

Iterators and Closures

Module 1.5 — Session 8 — 2026-03-24

The Iterator Trait

One required method, everything else is built on top:

```
trait Iterator {  
    type Item; // associated type – implementor chooses  
    fn next(&mut self) -> Option<Self::Item>; // Some(item) or None (exhausted)  
}
```

- `&mut self` — iterators are stateful cursors, not pure values
- `Option` signals exhaustion — no separate `hasNext()`/`next()` pair
- `type Item` — associated type, fixed per implementation (not chosen by caller)

Associated Types vs Generics

Associated types are chosen by the **implementor**, not the caller:

```
impl Iterator for GridIter {  
    type Item = Tile; // fixed – GridIter always yields Tiles  
}
```

A generic parameter `trait Iterator<Item>` would let one type implement the trait multiple times with different items. The associated type version enforces exactly one implementation per type.

Haskell analogy: associated types = type families inside a typeclass. Generic trait parameters = multi-parameter typeclasses.

Bounded Associated Types

Associated types can carry trait bounds — constraints on the implementor's choice:

```
trait IntoIterator {  
    type Item;  
    type IntoIter: Iterator<Item = Self::Item>; // must be an Iterator  
    fn into_iter(self) -> Self::IntoIter; // with matching Item  
}
```

`Self` in a trait definition = “the type implementing this trait.” You already know it from `&self` being sugar for `self: &Self`.

IntoIterator and for Loops

`for` loops desugar to `IntoIterator`:

```
for tile in &grid { ... }

// becomes (roughly):
let mut iter = (&grid).into_iter();
while let Some(tile) = iter.next() { ... }
```

`Vec<T>` implements `IntoIterator` three ways:

Call	Yields	Consumes?
<code>vec.into_iter()</code>	owned <code>T</code>	Yes
<code>(&vec).into_iter()</code>	<code>&T</code>	No
<code>(&mut vec).into_iter()</code>	<code>&mut T</code>	No

Every `Iterator` automatically implements `IntoIterator` via a blanket impl (returns itself). So iterators work directly in `for` loops.

Lazy Adapters and Consumers

Adapters (`.filter()`, `.map()`) return new iterator structs — no work happens. Consumers (`.sum()`, `.collect()`, `.count()`) drive the chain by calling `next()` repeatedly.

```
let result: i32 = numbers.iter()
    .filter(|n| **n % 2 == 0)
    .map(|n| n * n)
    .sum(); // ← consumer drives everything
```

filter’s Double Reference

`.iter()` on `Vec<i32>` yields `&i32`. But `filter` passes `&Item` to its predicate (it needs to keep items to yield later), so the closure gets `&&i32`.

Operators don’t auto-deref through multiple levels (unlike method calls, which chain arbitrarily). So `**n % 2` or pattern destructuring `|&&n| n % 2` is needed.

Key Insight

Method calls use a multi-step deref search (`T`, `&T`, `*T`, `**T...`). Operators just look for a direct trait impl — no deref chain. That’s why `&&i32 % 2` fails but `(&&i32).some_method()` could work.

Closures

Anonymous functions that capture variables from their environment:

```
|x| x * 2           // one arg, expression body
|x, y| x + y       // two args
|x| { ...; expr }  // block body
```

The name “closure” comes from lambda calculus: a lambda with free variables is “open”; capturing those variables from the environment **closes** it.

Capture Modes

The compiler infers the **least restrictive** capture mode:

Mode	When
<code>&T</code> (immutable borrow)	Closure only reads the variable
<code>&mut T</code> (mutable borrow)	Closure modifies the variable
Move (ownership)	Closure consumes or needs to own the variable

```
let mut count = 0;
let mut inc = || { count += 1; }; // captures &mut count
inc(); inc();
println!("{count}"); // still works – count was borrowed, not moved
```

The move keyword forces ownership capture regardless of what the body does:

```
let name = String::from("Bob");
let greet = move || println!("Hello, {name}");
// name is now moved into the closure – can't use it here
```

Essential when closures outlive their creating scope (e.g., thread spawning).

The Three Closure Traits

The compiler generates a hidden struct for each closure (captured variables = fields). The trait determines how the call method receives that struct:

Trait	Receives	Meaning
<code>Fn</code>	<code>&self</code>	Read-only captures, call many times
<code>FnMut</code>	<code>&mut self</code>	May mutate captures, call many times
<code>FnOnce</code>	<code>self</code>	May consume captures, call only once

Hierarchy: `Fn` $\subset$ `FnMut` $\subset$ `FnOnce` — every `Fn` is also `FnMut` is also `FnOnce`.

The compiler infers which trait a closure implements from what it does with captures:

- Only reads $\rightarrow$ `Fn`
- Mutates $\rightarrow$ `FnMut`
- Consumes (e.g., drop) $\rightarrow$ `FnOnce`

Reading API signatures

You rarely think about `Fn`/`FnMut`/`FnOnce` when **writing** closures — the compiler infers it. They matter when **reading** function signatures: `filter` takes `FnMut` (needs to call the predicate repeatedly), a one-shot callback might take `FnOnce`.